

Архитектурный бэкенд-план: Аналог «Яндекс Книг»

Техническое задание на разработку REST API на Django + PostgreSQL с интеграцией 2FA через Gmail

1. Архитектура приложений (Django Apps)

Для соблюдения модульности и чистоты кодовой базы, проект разделяется на четыре независимых приложения. Каждое приложение отвечает строго за свой изолированный слой бизнес-логики:

Приложение	Основное назначение	Ключевой фокус
users	Управление учетными записями, профилями пользователей и расширенными данными читателей.	Кастомный User, Profile
auth_2fa	Обеспечение безопасности входа: генерация, отправка одноразовых кодов на email и верификация сессий.	Временные токены, SMTP
books	Каталогизация произведений, авторов и жанров. Управление защищенными файлами книг.	Права админа, Хранилище
progress	Интерактивное взаимодействие читателя с контентом: сохранение позиции чтения, закладки и полки.	Связи Many-to-Many, Оптимизация

2. Спецификация моделей базы данных (ORM)

Приложение users

- **Кастомная модель Пользователя (User):** Расширяет базовый класс Django. Поле `email` используется в качестве уникального идентификатора вместо `username`. Содержит хэшированный пароль, дату создания и системные флаги прав доступа.
- **Модель Профиля (Profile):** Связана с моделью User через `OneToOneField`. Отвечает за пользовательские метаданные (аватар, краткая биография, настройки интерфейса ридера и текущий статус подписки).

Приложение auth_2fa

- **Модель Кода Подтверждения (EmailVerificationCode):** Таблица для контроля двухфакторного входа. Хранит связь с запрашивающим пользователем, сгенерированный 6-значный код, метку времени `created_at` и булев флаг `is_used`.

Важная бизнес-логика:

При каждой проверке кода бэкэнд вычисляет разницу времени между текущим моментом и полем `created_at`. Если интервал превышает 5 минут или флаг `is_used` равен `True`, код автоматически признается недействительным.

Приложение books

- **Модель Автор (Author):** Базовая информация о писателях (ФИО, фото, биографическая справка). Связь с книгами как один-ко-многим.
- **Модель Жанр (Genre):** Древовидная или плоская структура категорий литературы (например, Направляющие теги, Программирование, Киберпанк).
- **Модель Книга (Book):** Центральный узел каталога. Хранит название, аннотацию, ссылки на автора и жанры (`ManyToManyField`), защищенный файл книги (EPUB/FB2), обложку и флаг модерации `is_visible`.

Приложение progress

- **Модель Прогресс Чтения (ReadingProgress):** Связывает конкретного Пользователя и конкретную Книгу. Хранит номер последней прочитанной главы, позицию/абзац и вычисляемый процент прогресса для синхронизации между устройствами.
- **Модель Книжная Полка (BookShelf):** Пользовательские коллекции («Хочу прочитать», «Избранное»). Имеет связь `ForeignKey` с создателем полки и связь `ManyToManyField` со списком добавленных книг.

3. Проектирование REST API (Эндпоинты)

Поскольку процесс аутентификации разделен на этапы (пароль + 2FA), а также полностью изолирован от фронтенда, взаимодействие строится на базе Stateless-запросов с использованием токенов.

Маршруты Аутентификации и Пользователей

- `POST /api/users/register/` — **Регистрация:** Принимает email и пароль. Создает запись в БД, но возвращает статус успешного создания без генерации авторизационного токена.
- `POST /api/auth/login/` — **Авторизация (Шаг 1):** Принимает email и пароль. Проверяет корректность данных. В случае успеха генерирует 6-значный код, отправляет его через SMTP на почту и возвращает статус `need_code` и временный токен сессии.

- `POST /api/auth/verify/` — **Верификация 2FA (Шаг 2)**: Принимает введенный код и временный токен. Проверяет валидность. Если код верный, генерирует финальный долгоживущий Access-Token (JWT) для всех последующих защищенных запросов.

Маршруты Каталога Книг и Прогресса

- `GET /api/books/` — **Список книг**: Публичный эндпоинт для отображения витрины библиотеки. Поддерживает фильтрацию по жанрам и авторам.
- `GET /api/books/<id>/` — **Детали книги**: Доступен только авторизованным пользователям с активным токеном. Отдает структурированный текст или защищенную ссылку на файл.
- `POST /api/books/` — **Добавление контента**: Закрытый эндпоинт. Бэкенд выполняет жесткую валидацию прав доступа: если `request.user.is_staff` равен `False`, запрос отклоняется со статусом 403 Forbidden.
- `POST /api/progress/update/` — **Синхронизация прогресса**: Фронтенд регулярно отправляет текущие координаты чтения (ID книги, страница) при перелистывании для перезаписи ``ReadingProgress``.

4. Очередность выполнения задач (План действий)

1. **Инициализация**: Создание проекта в Django, подключение базы данных PostgreSQL вместо дефолтной SQLite.
2. **Моделирование**: Генерация 4-х приложений, проектирование полей моделей и связей, выполнение первоначальных миграций.
3. **Администрирование**: Инициализация суперпользователя системы через команду `createsuperuser`, кастомизация панели управления под модель Book.
4. **Интеграция Почты**: Настройка SMTP-клиента в ``settings.py`` (Gmail App Password) и тестирование отправки писем в консольном режиме.
5. **Разработка API**: Написание кастомной логики двухэтапного Login View и сериализаторов DRF.